

ROOM DATABASE

1. Tổng quan (Overview)

Room là một thư viện trừu tượng (abstraction layer) nằm trên SQLite. Nó giúp bạn tận dụng toàn bộ sức mạnh của SQLite trong khi vẫn đảm bảo kiểm tra lỗi compile-time và cú pháp gọn nhẹ hơn.

Room is an abstraction layer over SQLite. It allows for more robust database access while harnessing the full power of SQLite, ensuring compile-time checks and cleaner code.

2. Các thành phần chính (Core Components)

Room bao gồm 3 thành phần cốt lõi mà bạn buộc phải nắm vững:

- **Entity:** Đại diện cho một bảng (table) trong CSDL. Mỗi field trong class tương ứng với một cột.
 - *Represents a table within the database. Each field in the class corresponds to a column.*
 - **DAO (Data Access Object):** Chứa các phương thức để truy cập CSDL (Insert, Update, Delete, Query). Đây là nơi bạn viết code SQL.
 - *Contains methods used for accessing the database. This is where you write your SQL queries.*
 - **Room Database:** Là điểm truy cập chính, đóng vai trò kết nối ứng dụng với SQLite.
 - *The main access point for the connection to your app's persisted data.*
-

3. Cách hoạt động (How it works)

1. **Định nghĩa thực thể (Define Entity):** Bạn tạo một data class và đánh dấu nó là `@Entity`.
 2. **Tạo DAO:** Bạn tạo một Interface và sử dụng các annotation như `@Query` hoặc `@Insert`.
 3. **Tạo Database class:** Bạn tạo một abstract class kế thừa `RoomDatabase`.
 4. **Sử dụng (Usage):** Room sẽ tự động sinh code thực thi dựa trên các annotation bạn cung cấp tại thời điểm biên dịch (compile-time).
-

4. Triển khai Code (Implementation)

Bước 1: Thêm Dependency (Add Dependencies)

Trong tệp `build.gradle` (Module: `app`):

Gradle

```
dependencies {  
    def room_version = "2.6.1"  
    implementation "androidx.room:room-runtime:$room_version"
```

```

annotationProcessor "androidx.room:room-compiler:$room_version"
// Cho Kotlin sử dụng Kapt hoặc KSP
kapt "androidx.room:room-compiler:$room_version"
}

```

Bước 2: Tạo Entity

Kotlin

```

@Entity(tableName = "users")
data class User(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    @ColumnInfo(name = "full_name") val name: String?,
    @ColumnInfo(name = "age") val age: Int
)

```

Bước 3: Tạo DAO

Kotlin

```

@Dao
interface UserDao {
    @Query("SELECT * FROM users")
    fun getAll(): List<User>

    @Insert
    fun insertAll(vararg users: User)

    @Delete
    fun delete(user: User)
}

```

Bước 4: Tạo Room Database

Kotlin

```

@Database(entities = [User::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun userDao(): UserDao
}

```

5. So sánh: SQLite vs Room (Comparison)

Tính năng (Feature)	SQLite thuần (Standard SQLite)	Room Database
------------------------	-----------------------------------	---------------

Compile-time check	Không (Dễ lỗi cú pháp)	Có (Báo lỗi ngay khi code)
Boilerplate code	Rất nhiều	Rất ít
Dễ sử dụng	Khó, tốn thời gian	Dễ, tích hợp tốt với LiveData/Flow
Migration	Thủ công và phức tạp	Có hỗ trợ công cụ Migration

Xuất sang Trang tính

6. Ứng dụng thực tế (Real-world Applications)

Room thường được sử dụng trong các kiến trúc hiện đại như **MVVM** hoặc **Clean Architecture** để làm nguồn dữ liệu cục bộ (Local Data Source).

- **Caching dữ liệu:** Lưu trữ dữ liệu từ API để người dùng có thể xem khi offline.
 - *Data Caching: Storing API data so users can view it offline.*
 - **Lưu trữ cài đặt/Trạng thái:** Lưu trạng thái ứng dụng hoặc danh sách yêu thích của người dùng.
 - *Storing settings/state: Saving user preferences or bookmarks.*
 - **Đồng bộ hóa:** Kết hợp với WorkManager để đồng bộ dữ liệu lên server sau khi đã lưu tạm ở Room.
 - *Synchronization: Working with WorkManager to sync local data to a server later.*
-

7. Lưu ý quan trọng (Important Notes)

- **Main Thread:** Room không cho phép truy cập CSDL trên Main Thread (luồng chính) vì sẽ gây giật lag (ANR). Bạn phải sử dụng **Coroutines** hoặc **RxJava**.
 - *Room doesn't allow database access on the main thread to avoid UI freezes. Use Coroutines or RxJava instead.*
- **Singleton Pattern:** Bạn nên khởi tạo instance của Room Database theo mô hình Singleton để tránh lãng phí tài nguyên.
 - *Use the Singleton pattern for the database instance to prevent resource leaks.*

LIVEDATA

1. Tổng quan (Overview)

LiveData là một lớp giữ dữ liệu có thể quan sát được (observable data holder class) và đặc biệt là nó **nhận biết vòng đời** (lifecycle-aware). Điều này có nghĩa là nó tôn trọng vòng đời của các thành phần khác như Activity, Fragment hoặc Service.

LiveData is an observable data holder class. Unlike a regular observable, LiveData is lifecycle-aware, meaning it respects the lifecycle of other app components, such as activities, fragments, or services.

2. Các đặc tính chính (Key Advantages)

- **Không lo rò rỉ bộ nhớ (No memory leaks):** Observer sẽ tự động hủy khi LifecycleOwner bị tiêu hủy.
 - *Observers are bound to Lifecycle objects and clean up after themselves.*
 - **Không crash do Activity dừng (No crashes due to stopped activities):** Nếu Activity nằm trong back stack, nó sẽ không nhận sự kiện từ LiveData.
 - *If the observer's lifecycle is inactive, it doesn't receive any LiveData events.*
 - **Dữ liệu luôn cập nhật (Data is always up to date):** Khi một thành phần trở lại trạng thái hoạt động (active), nó sẽ nhận được dữ liệu mới nhất ngay lập tức.
 - *If a lifecycle becomes active again, it receives the latest data automatically.*
 - **Tách biệt Logic và UI (Separation of concerns):** Giúp triển khai mô hình MVVM hiệu quả hơn.
 - *Facilitates the MVVM pattern by decoupling the UI from the data source.*
-

3. Các thành phần và Cách hoạt động (Components & How it works)

MutableLiveData vs LiveData

- **LiveData:** Chỉ đọc (Read-only). Dùng để expose dữ liệu từ ViewModel ra View.
- **MutableLiveData:** Có thể thay đổi (Editable). Cung cấp các phương thức setValue() và postValue().

Cách cập nhật dữ liệu (Updating Data)

1. **setValue():** Sử dụng khi bạn đang ở **Main Thread**. Cập nhật ngay lập tức.
 2. **postValue():** Sử dụng khi bạn đang ở **Background Thread**. Nó sẽ chuyển tác vụ về Main Thread để cập nhật.
-

4. Triển khai Code (Implementation)

Dưới đây là ví dụ kết hợp LiveData trong mô hình MVVM:

Bước 1: Trong ViewModel

Kotlin

```
class UserViewModel : ViewModel() {  
    // Luồng dữ liệu nội bộ có thể sửa đổi  
    private val _userName = MutableLiveData<String>()  
  
    // Luồng dữ liệu công khai chỉ có thể đọc  
    val userName: LiveData<String> get() = _userName  
  
    fun updateName(newName: String) {
```

```

        _userName.value = newName // Dùng trên Main Thread
    }
}

```

Bước 2: Trong Activity/Fragment (View)

Kotlin

```

class UserActivity : AppCompatActivity() {
    private lateinit var viewModel: UserViewModel

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        viewModel = ViewModelProvider(this).get(UserViewModel::class.java)

        // Bắt đầu quan sát (Observe)
        viewModel.userName.observe(this, Observer { name ->
            // Cập nhật UI mỗi khi dữ liệu thay đổi
            textView.text = name
        })
    }
}

```

5. Ứng dụng thực tế (Real-world Applications)

LiveData thường đóng vai trò là "cầu nối" giữa dữ liệu (Repository/Room) và giao diện:

- **Phản hồi từ Database:** Kết hợp với **Room Database** để tự động cập nhật UI khi dữ liệu trong bảng thay đổi.
 - *Combined with Room to automatically update UI when database tables change.*
- **Trạng thái API:** Theo dõi trạng thái Loading, Success, hoặc Error khi gọi API.
 - *Monitoring API status: Loading, Success, or Error states.*
- **Chia sẻ dữ liệu giữa các Fragment:** Sử dụng một Shared ViewModel chứa LiveData để các Fragment trong cùng một Activity giao tiếp với nhau.
 - *Sharing data between Fragments using a Shared ViewModel.*

6. LiveData vs StateFlow/SharedFlow

Trong các dự án sử dụng Kotlin Coroutines hiện đại, nhiều lập trình viên đang chuyển dần sang **StateFlow**.

Feature	LiveData	StateFlow

Lifecycle Awareness	Có sẵn (Native)	Cần sử dụng repeatOnLifecycle
Initial Value	Không bắt buộc	Bắt buộc
Thread	Gắn chặt với Main Thread	Linh hoạt hơn (Flow-based)

7. Lưu ý quan trọng (Important Notes)

- 1. **Luôn observe trong onCreate():** Điều này đảm bảo hệ thống không tạo ra các lời gọi quan sát dư thừa mỗi khi Activity được khôi phục.
- 2. **Đừng quên ViewLifecycleOwner:** Trong Fragment, hãy sử dụng viewLifecycleOwner thay vì this để tránh lỗi rò rỉ khi Fragment bị destroy nhưng instance vẫn còn.

RECYCLE VIEW

1. Tổng quan (Overview)

RecyclerView là một widget dùng để hiển thị các danh sách dữ liệu lớn một cách hiệu quả bằng cách "tái sử dụng" (recycle) các view đã bị cuộn ra khỏi màn hình.

RecyclerView is a flexible and efficient widget for displaying large data sets by recycling views that have scrolled off the screen, significantly improving performance and memory usage.

2. Các thành phần cốt lõi (Core Components)

Để RecyclerView hoạt động, bạn cần 4 thành phần chính:

- **Adapter:** Bộ trung gian kết nối dữ liệu với View. Nó tạo ra các ViewHolder và gán dữ liệu cho chúng.
 - *The bridge between the data and the View. It creates ViewHolders and binds data to them.*
- **ViewHolder:** Một wrapper chứa các view thành phần của một item (như TextView, ImageView). Giúp tránh việc gọi findViewById() liên tục.
 - *A wrapper around a view that contains the layout for an individual item in the list.*
- **LayoutManager:** Quyết định cách các item được sắp xếp (Dạng hàng dọc, hàng ngang, hay lưới).
 - *Positions item views inside the RecyclerView and determines when to reuse item views.*
- **ItemAnimator:** Xử lý các hiệu ứng hoạt họa khi thêm, xóa hoặc di chuyển item.
 - *Animates the appearance, disappearance, and movement of items.*

3. Các loại LayoutManager phổ biến (Common LayoutManagers)

1. **LinearLayoutManager:** Hiển thị danh sách theo hàng dọc hoặc hàng ngang.
 2. **GridLayoutManager:** Hiển thị danh sách dưới dạng lưới (Grid).
 3. **StaggeredGridLayoutManager:** Hiển thị dạng lưới so le (giống Pinterest).
-

4. Triển khai Code (Implementation)

Bước 1: Khai báo trong Layout (XML)

XML

```
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/recyclerView"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

Bước 2: Tạo Adapter và ViewHolder

Kotlin

```
class MyAdapter(private val dataList: List<String>) :
    RecyclerView.Adapter<MyAdapter.MyViewHolder>() {

    // ViewHolder: Giữ các reference đến View
    class MyViewHolder(view: View) : RecyclerView.ViewHolder(view) {
        val textView: TextView = view.findViewById(R.id.itemText)
    }

    // Tạo View mới (invoked by the layout manager)
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MyViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_layout, parent, false)
        return MyViewHolder(view)
    }

    // Gán dữ liệu vào View (invoked by the layout manager)
    override fun onBindViewHolder(holder: MyViewHolder, position: Int) {
        holder.textView.text = dataList[position]
    }

    override fun getItemCount() = dataList.size
}
```

Bước 3: Thiết lập trong Activity/Fragment

Kotlin

```
val recyclerView: RecyclerView = findViewById(R.id.recyclerView)
recyclerView.layoutManager = LinearLayoutManager(this)
recyclerView.adapter = MyAdapter(myDataset)
```

5. Ứng dụng thực tế (Real-world Applications)

- **Bảng tin mạng xã hội (Social Media Feeds):** Hiển thị danh sách bài đăng với nhiều loại layout khác nhau.
 - *Displaying news feeds with complex, multi-type layouts.*
- **Danh bạ/Danh sách tin nhắn:** Quản lý hàng ngàn bản ghi mà không tốn nhiều RAM.
 - *Managing thousands of contacts or messages with minimal memory overhead.*
- **Thư viện ảnh (Gallery):** Sử dụng GridLayoutManager để hiển thị ảnh.
 - *Using GridLayoutManager to display image collections.*

6. Mẹo tối ưu hóa (Optimization Tips)

- **DiffUtil:** Thay vì gọi notifyDataSetChanged() (làm mới toàn bộ), hãy dùng DiffUtil để chỉ cập nhật những item có sự thay đổi. Điều này giúp tăng hiệu năng cực lớn.
 - *Use DiffUtil to calculate the difference between two lists and update only the changed items.*
- **setHasFixedSize(true):** Nếu bạn biết kích thước của RecyclerView không thay đổi theo nội dung, hãy bật cái này để tối ưu việc đo đạc layout.
 - *If the size of the RecyclerView itself doesn't change, enabling this improves performance.*

7. So sánh: ListView vs RecyclerView

Đặc điểm (Feature)	ListView	RecyclerView
ViewHolder pattern	Tùy chọn (Optional)	Bắt buộc (Compulsory)
Layout	Chỉ dọc (Vertical)	Đa dạng (Vertical, Horizontal, Grid)
Animation	Khó triển khai	Có sẵn và rất mượt mà
Hiệu năng	Kém hơn khi dữ liệu lớn	Tối ưu hóa cực tốt

3 Layer Architecture

1. Tổng quan (Overview)

3-Layer Architecture (còn gọi là Multitier Architecture) chia ứng dụng thành 3 lớp: Presentation, Business Logic, và Data Access. Mỗi lớp chỉ giao tiếp với lớp ngay bên dưới nó.

3-Layer Architecture organizes an application into three logical layers: Presentation, Business Logic, and Data Access. Each layer has a specific responsibility and communicates only with the layer directly below it.

2. Các lớp thành phần (The Three Layers)

A. Lớp Giao diện (Presentation Layer - UI)

Đây là lớp trên cùng, nơi người dùng tương tác với ứng dụng.

- **Trách nhiệm:** Hiển thị dữ liệu cho người dùng và nhận dữ liệu đầu vào. Nó không chứa logic nghiệp vụ phức tạp.
- **Thành phần:** Activity, Fragment (Android), HTML/CSS (Web), hoặc các UI Component.

Responsibility: Handles user interaction and data display. It gathers input from the user and passes it to the lower layers.

B. Lớp Nghiệp vụ (Business Logic Layer - BLL)

Đóng vai trò là "bộ não" của ứng dụng, nằm giữa lớp UI và Data.

- **Trách nhiệm:** Xử lý các quy tắc nghiệp vụ, tính toán, xác thực dữ liệu (validation) và điều hướng luồng dữ liệu.
- **Thành phần:** Services, Use Cases, hoặc Business Objects.

Responsibility: Acts as the "brain" of the application. It processes business rules, performs calculations, and coordinates data flow.

C. Lớp Truy cập Dữ liệu (Data Access Layer - DAL)

Lớp dưới cùng, chịu trách nhiệm giao tiếp với các nguồn lưu trữ.

- **Trách nhiệm:** Thực hiện các thao tác CRUD (Create, Read, Update, Delete) trên cơ sở dữ liệu hoặc gọi các External API.
- **Thành phần:** Room Database, Retrofit, DAO, Repository.

Responsibility: Manages data persistence. It communicates with databases, file systems, or external web services.

3. Luồng hoạt động (Data Flow)

Luồng đi của dữ liệu thường tuân theo sơ đồ:

1. User nhập liệu ở **Presentation Layer**.
2. **Presentation Layer** gọi hàm ở **Business Layer**.
3. **Business Layer** xử lý logic rồi yêu cầu **Data Layer** lưu trữ/lấy dữ liệu.
4. Dữ liệu trả về theo chiều ngược lại.

4. Tại sao nên dùng 3-Layer? (Benefits)

- **Tính bảo trì (Maintainability):** Khi bạn thay đổi CSDL (ví dụ từ SQL sang MongoDB), bạn chỉ cần sửa lớp **Data Access** mà không làm ảnh hưởng đến giao diện.
 - *Changes in one layer (e.g., switching databases) have minimal impact on others.*
- **Khả năng mở rộng (Scalability):** Dễ dàng thêm tính năng mới vào lớp nghiệp vụ mà không làm rối mã nguồn UI.
 - *Easier to add new features or scale specific parts of the application.*
- **Dễ kiểm thử (Testability):** Bạn có thể viết Unit Test cho lớp Business mà không cần chạy giao diện hay kết nối CSDL thật.
 - *Enables easier unit testing by isolating business logic from UI and DB.*

5. Ví dụ minh họa (Code Example - Kotlin)

Giả sử bạn làm chức năng đăng ký thành viên:

1. **Data Layer (UserDao):** Chịu trách nhiệm lưu User vào DB.
2. **Business Layer (UserService):** Kiểm tra xem Email đã tồn tại chưa, mật khẩu có đủ độ dài không.
3. **Presentation Layer (RegisterActivity):** Nhận text từ EditText và gọi UserService.

Kotlin

// Data Access Layer

```
class UserRepository {  
    fun saveUser(user: User) { /* Code lưu vào DB */ }  
}
```

// Business Logic Layer

```
class UserService(private val repo: UserRepository) {  
    fun register(user: User): Boolean {  
        if (user.email.isEmpty()) return false // Logic nghiệp vụ  
        repo.saveUser(user)  
        return true  
    }  
}
```

// Presentation Layer

```
class RegisterActivity : AppCompatActivity() {  
    private val userService = UserService(UserRepository())  
  
    fun onRegisterClicked() {  
        val user = User(emailInput.text.toString())  
        val success = userService.register(user)  
        if (success) showToast("Thành công")  
    }  
}
```

6. So sánh với Clean Architecture

3-Layer là tiền thân của nhiều kiến trúc hiện đại. Tuy nhiên, trong các dự án lớn, người ta thường dùng **Clean Architecture** hoặc **Hexagonal Architecture** để tách biệt sâu hơn nữa (ví dụ: tách Repository Pattern ra khỏi Business Logic bằng Interface).

Đặc điểm	3-Layer Architecture	Clean Architecture
Độ phức tạp	Thấp/Trung bình	Cao
Sự phụ thuộc	UI -> Business -> Data	Tất cả hướng về Entities (Center)
Dự án phù hợp	Nhỏ và vừa	Lớn, phát triển lâu dài